

Software Requirements Specification (SRS) for SENA Network

Document Version: 1.1.0

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to detail the functional and non-functional requirements for the SENA Network, a Layer 2 (L2) modular infrastructure. This document serves as a foundational guide for development, testing, and deployment, ensuring all stakeholders have a clear understanding of the system to be built.

1.2 Scope

The SENA Network will provide a high-performance, secure, and user-friendly L2 blockchain solution built on the Sovereign SDK (Rust) and leveraging RocksDB for state management. It will integrate with the Aptos Layer 1 (L1) for security settlement and will feature protocol-level support for keyless authentication, social identity mapping, multi-asset gas payments, optimistic governance, and an interactive fraud proof system that makes the optimistic security model enforceable by any party. This SRS covers the requirements for the core L2 protocol and its primary modules.

1.3 Definitions, Acronyms, and Abbreviations

- **L2:** Layer 2 blockchain, built on top of an existing Layer 1 blockchain.
- **L1:** Layer 1 blockchain, the base layer for security and finality (in this case, Aptos).
- **SENA:** The name of the Layer 2 network.
- **Sovereign SDK:** A framework for building modular rollups in Rust.
- **RocksDB:** A high-performance embedded key-value store.
- **NOMT:** Nearly-Optimal Merkle Trie, an optimized Merkle tree implementation.
- **Aptos:** The Layer 1 blockchain providing security and settlement for SENA.
- **OIDC:** OpenID Connect, an authentication layer on top of OAuth 2.0.
- **JWT:** JSON Web Token, a compact, URL-safe means of representing claims between two parties.

- **ZKP:** Zero-Knowledge Proof, a method by which one party (the prover) can prove to another party (the verifier) that a given statement is true, without conveying any information apart from the fact that the statement is indeed true.
- **EPK:** Ephemeral Public Key.
- **PRD:** Product Requirements Document.
- **SRS:** Software Requirements Specification.
- **SDD:** Software Design Description.
- **STF:** State Transition Function. The deterministic function mapping a pre-state and an ordered batch of transactions to a post-state.
- **Assertion:** A bonded claim, submitted to Aptos L1, that executing a specific batch against a specific pre-state root yields a specific post-state root.
- **Challenge Window:** The period during which a pending assertion may be disputed. An assertion that survives it becomes finalized.
- **Bisection:** The interactive protocol by which two disagreeing parties binary-search an execution trace to isolate the single step on which they first diverge.
- **OSP:** One-Step Proof. A proof that a single, isolated instruction of the STF was executed correctly, small enough to be verified on L1.
- **DA:** Data Availability. The guarantee that the transaction data behind an assertion is published and retrievable, so that any party can independently re-execute it.
- **Verifier Node:** A node that re-executes every batch and challenges any assertion whose post-state root disagrees with its own.
- **Honest Verifier Assumption (1-of-N):** The assumption that at least one honest party runs a verifier node and is able to submit a challenge before the challenge window expires.

1.4 References

- [1] SENA Network: Layer 2 Modular Infrastructure Documentation (Previous Stakeholder Document)
- [2] Sovereign SDK. Sovereign Labs. Available at: <https://www.sovereign.xyz/>
- [3] RocksDB. GitHub. Available at: <https://github.com/facebook/rocksdb>
- [4] Aptos Keyless Accounts. Aptos Docs. Available at: <https://aptos.dev/concepts/accounts/#keyless-accounts>
- [5] Software Design Description (SDD) for SENA Network, Section 3.5 (Fraud Proof and Dispute Resolution System)

2. Overall Description

2.1 Product Perspective

SENA Network is a standalone L2 blockchain that operates as a client-server application, with its security and data availability rooted in the Aptos L1. It is not a smart contract on Aptos L1 but rather an independent chain that periodically commits its state to Aptos L1. This architecture allows SENA to achieve high transaction throughput and low latency while inheriting the robust security guarantees of Aptos.

SENA is an **optimistic** rollup: state roots are accepted as valid by default and are subject to challenge for a bounded window. The fraud proof system specified in Section 3.1.6 is what converts that optimism into a security guarantee. Without it, a committed state root is only as trustworthy as the sequencer that produced it; with it, a single honest verifier anywhere in the world can prevent an invalid state root from ever finalizing.

2.2 Product Functions

The primary functions of the SENA Network include:

- Processing and executing L2 transactions.
- Managing user accounts with Web2-style keyless authentication.
- Mapping human-readable social identifiers to blockchain addresses.
- Facilitating gas payments using stablecoins.
- Supporting an optimistic governance mechanism.
- Committing L2 state roots to Aptos L1 for security and finality.
- Publishing batch transaction data so that any party can independently re-execute the chain.
- Resolving disputes over committed state roots through an interactive fraud proof protocol adjudicated on Aptos L1.

2.3 User Characteristics

- **End-Users:** Expect a seamless, intuitive experience akin to Web2 applications, without needing to manage seed phrases or native gas tokens. They will interact with applications built on SENA.
- **Application Developers:** Require a robust, well-documented, and easy-to-integrate platform for building consumer-facing dApps. They will interact with SENA's RPC endpoints and SDKs.
- **Node Operators:** Responsible for running SENA sequencer nodes. They should have basic system administration skills and an understanding of blockchain node operation.
- **Verifiers (Challengers):** Independent parties — which may include application developers, node operators, exchanges, large token holders, or public-good watchtower services — who run verifier nodes

to re-execute the chain and challenge invalid assertions. They require no permission, allowlisting, or relationship with the sequencer operator.

2.4 General Constraints

- **Technology Stack:** Must be built using Rust and Sovereign SDK.
- **Database:** Must utilize RocksDB for persistent storage, optimized with NOMT.
- **L1 Integration:** Must integrate with Aptos L1 for security settlement and state anchoring.
- **Performance:** Must support high transaction throughput suitable for consumer applications.
- **Security:** Must achieve "Aptos-Level Secure" as defined in the stakeholder documentation.
- **Resource Efficiency:** Must be capable of running on low-end server hardware for sequencer nodes.
- **Determinism:** The STF must be bit-for-bit reproducible across machines, compilers, and platforms. Any nondeterminism makes honest parties disagree and renders fraud proofs unsound.
- **L1 Gas Budget:** One-step verification must execute within the per-transaction gas and computation limits of the Aptos L1 Move VM.

2.5 Assumptions and Dependencies

- Aptos L1 remains a stable and secure blockchain.
- The Aptos public, decentralized ZK prover network is available and reliable for offloading ZKP generation.
- Sovereign SDK continues to be actively developed and maintained.
- Rust ecosystem tools and libraries remain stable and compatible.
- **At least one honest party runs a verifier node** and retains the ability to submit a challenge transaction to Aptos L1 within the challenge window. The security of the fraud proof system rests entirely on this assumption.
- **Aptos L1 does not censor** challenge transactions for the duration of the challenge window.

3. Specific Requirements

3.1 Functional Requirements

3.1.1 Module 1: Protocol-Enforced Keyless Authentication Engine

- **REQ-AUTH-001:** The system SHALL natively process OpenID Connect (OIDC) identities as standard account signers at the protocol layer.
- **REQ-AUTH-002:** The system SHALL derive deterministic L2 addresses using an application-scoped

identifier (\$aud), a user identifier (\$sub), and an obfuscating pepper value.

- **REQ-AUTH-003:** The system SHALL accept transactions signed by a short-lived Ephemeral Public Key (EPK).
- **REQ-AUTH-004:** The system SHALL verify the validity of an EPK against a Zero-Knowledge Proof (ZKP) that references a valid Web2 JWT signature from an approved Identity Provider (e.g., Google or Apple OAuth).
- **REQ-AUTH-005:** The system SHALL NOT calculate heavy ZK proof constraints locally; it SHALL ingest pre-computed proofs generated by external client-side utilities or decentralized prover networks.
- **REQ-AUTH-006:** The system SHALL perform lightweight cryptographic verification checks of ZK proofs during block execution.
- **REQ-AUTH-007:** ZK proof verification SHALL be implemented as a deterministic, side-effect-free routine within the STF, so that it can be re-executed identically by verifier nodes and, if disputed, adjudicated by the one-step verifier.

3.1.2 Module 2: The Social Connect Identity Index

- **REQ-SOCIAL-001:** The system SHALL maintain an embedded system ledger connecting human-readable identifiers directly to SENA network addresses.
- **REQ-SOCIAL-002:** The system SHALL store identifiers as unique cryptographic hashes (SHA-256(Raw Identifier + Network Salt)) to ensure privacy.
- **REQ-SOCIAL-003:** The system SHALL NOT expose plain text personal information (phone numbers, emails, handles) directly in the global state database.
- **REQ-SOCIAL-004:** The system SHALL provide an RPC endpoint for external financial or payment software to perform lookups to resolve a destination address instantly from a human-readable identifier.
- **REQ-SOCIAL-005:** The system SHALL support mapping a single SENA address to multiple verified Web2 social channels simultaneously.

3.1.3 Module 3: Multi-Asset Gas Paymaster Engine

- **REQ-GAS-001:** The system SHALL support transaction gas payment via an approved whitelist of stablecoins.
- **REQ-GAS-002:** The system SHALL remove the requirement for users to hold a native network utility token for routine actions.
- **REQ-GAS-003:** The network state machine SHALL handle an adjustable system ledger tracking permitted gas assets (e.g., native bridged USDC, USDT, and EUR-equivalent stablecoins).
- **REQ-GAS-004:** When an application transaction is processed, the sequencer SHALL evaluate the current gas cost.

- **REQ-GAS-005:** The sequencer SHALL look up the real-time fee rate for the chosen stablecoin.
- **REQ-GAS-006:** The sequencer SHALL extract the fraction of a penny directly out of the user's transaction payload or account balance as payment.
- **REQ-GAS-007:** Collected stablecoin fees SHALL be directed into a system-controlled vault.
- **REQ-GAS-008:** The sequencer SHALL handle converting a portion of collected assets into native APT tokens via the Aptos L1 bridge contract.
- **REQ-GAS-009:** The system SHALL continuously fund the L2's cryptographic checkpoint commitments on the Layer 1 mainnet using converted APT tokens.
- **REQ-GAS-010:** The exchange rate used for fee conversion SHALL be read from in-state values updated through an auditable, replayable mechanism, such that the rate applied to any historical transaction is deterministically reconstructible by a verifier node.

3.1.4 Module 4: Optimistic L2 Governance Layer

- **REQ-GOV-001:** The system SHALL support an off-chain aggregation mechanism for token holders to vote on parameter revisions by staking the network's asset (\$SENA).
- **REQ-GOV-002:** Once a vote passes off-chain, the structured result payload SHALL be signed by a designated Governance Multi-Signature Council.
- **REQ-GOV-003:** The SENA node runtime SHALL include an administrative module to ingest the council-signed payload.
- **REQ-GOV-004:** The administrative module SHALL authenticate the cryptographic signatures in milliseconds.
- **REQ-GOV-005:** The administrative module SHALL instantly apply updates to system variables (e.g., whitelisting new stablecoins, adjusting network fee schedules) based on authenticated payloads.
- **REQ-GOV-006:** Governance parameter updates SHALL take effect as ordinary state transitions within a batch, so that they are covered by the same fraud proof guarantees as user transactions.
- **REQ-GOV-007:** Governance SHALL NOT be able to finalize a pending assertion, discard a challenge, alter the outcome of a dispute, or reduce the challenge window below the hard floor defined in REQ-FRAUD-004. These operations SHALL be unreachable from the administrative module by construction, not by policy.

3.1.5 Module 5: Fraud Proof and Dispute Resolution System

Assertions and bonding

- **REQ-FRAUD-001:** Every commitment to Aptos L1 SHALL take the form of an *assertion* containing, at minimum: the L2 block height range covered, the pre-state root, the post-state root, a commitment to the ordered batch data, the transaction count, and the identity of the proposer.

- **REQ-FRAUD-002:** Each assertion SHALL form a chain, referencing its parent assertion by hash, such that an assertion is valid only if its pre-state root equals its parent's post-state root.
- **REQ-FRAUD-003:** A proposer SHALL post a bond of at least `SEQUENCER_BOND_MIN` before an assertion is accepted. The bond SHALL remain locked until the assertion and all of its descendants are finalized.
- **REQ-FRAUD-004:** An assertion SHALL remain in the Pending state for the duration of `CHALLENGE_WINDOW` before transitioning to Finalized. `CHALLENGE_WINDOW` SHALL be a governance-adjustable parameter with a default of seven (7) days and a hard lower bound of twenty-four (24) hours enforced by the L1 contract.
- **REQ-FRAUD-005:** The L1 bridge contract SHALL honour withdrawals only against state proofs rooted in a Finalized assertion.
- **REQ-FRAUD-006:** An assertion SHALL NOT finalize while any unresolved challenge against it, or against any of its ancestors, remains open.

Data availability

- **REQ-FRAUD-007:** For every assertion, the proposer SHALL publish the complete, ordered transaction data of the batch, sufficient for any party to deterministically re-execute the batch and reproduce the claimed post-state root.
- **REQ-FRAUD-008:** Batch data SHALL be published to Aptos L1 in a form that inherits L1 data availability guarantees, or to an approved data availability layer whose commitment is recorded in the assertion.
- **REQ-FRAUD-009:** Any party SHALL be able to raise a *data availability challenge* against a pending assertion. If the proposer does not publish the referenced batch data within `DA_RESPONSE_TIMEOUT`, the assertion SHALL be rejected and the proposer's bond slashed.
- **REQ-FRAUD-010:** The system SHALL NOT permit an assertion to finalize on the basis of data the challenger community cannot obtain. Unavailable data SHALL be treated as invalid data.

Challenging and dispute resolution

- **REQ-FRAUD-011:** Any party SHALL be able to challenge a Pending assertion permissionlessly, without allowlisting, registration, or approval, by posting a bond of at least `CHALLENGER_BOND`.
- **REQ-FRAUD-012:** A dispute SHALL be resolved by an interactive bisection protocol in which the two parties binary-search the execution trace of the disputed batch to isolate the single step at which their claimed intermediate states first diverge.
- **REQ-FRAUD-013:** The bisection protocol SHALL terminate in $O(\log N)$ rounds for a trace of N steps.
- **REQ-FRAUD-014:** Each move in the bisection protocol SHALL be bounded by `MOVE_TIMEOUT`. A

party that fails to make a valid move before its timeout expires SHALL forfeit the dispute.

- **REQ-FRAUD-015:** Each party SHALL be allotted a per-dispute time budget (a chess clock). Timeout enforcement SHALL account for L1 congestion so that an honest party is not defeated by an inability to land a transaction.
- **REQ-FRAUD-016:** Once bisection isolates a single step, the challenger SHALL submit a One-Step Proof (OSP) consisting of the instruction, the pre-step machine state commitment, the post-step machine state commitment, and NOMT inclusion proofs for every state element the step reads or writes.
- **REQ-FRAUD-017:** The L1 contract SHALL execute the single disputed instruction and compare its result against the post-step commitment, resolving the dispute without any trusted third party.
- **REQ-FRAUD-018:** One-step verification on L1 SHALL be self-contained: it SHALL depend on no data beyond the OSP, the disputed step's commitments, and values already recorded on L1.

Outcomes

- **REQ-FRAUD-019:** When a challenge succeeds, the disputed assertion and all of its descendants SHALL be discarded, and the canonical L2 state SHALL roll back to the most recent finalized assertion.
- **REQ-FRAUD-020:** When a challenge succeeds, the losing proposer's bond SHALL be slashed. A portion SHALL be paid to the successful challenger as a reward sufficient to cover verification and L1 gas costs; the remainder SHALL be directed to the network treasury rather than to the challenger, to limit the profitability of collusive self-challenge.
- **REQ-FRAUD-021:** When a challenge fails, the challenger's bond SHALL be slashed and distributed on the same basis, so that frivolous challenges carry a real cost.
- **REQ-FRAUD-022:** A proposer whose assertion is successfully challenged SHALL be removed from the set of permitted proposers until reinstated by governance.
- **REQ-FRAUD-023:** Following a rollback, the system SHALL re-derive the L2 state from the last finalized assertion and the published batch data, and SHALL expose the set of reverted L2 blocks through the RPC API.

Verifier nodes

- **REQ-FRAUD-024:** The SENA node software SHALL provide a verifier mode that follows the chain, re-executes every batch from published data, and compares the result against each assertion.
- **REQ-FRAUD-025:** A verifier node SHALL automatically open a challenge upon detecting a divergence, and SHALL be capable of playing the bisection protocol to completion without operator intervention.
- **REQ-FRAUD-026:** A verifier node SHALL be runnable on commodity hardware, at a cost low enough that independent operation is economically realistic for ordinary ecosystem participants.

- **REQ-FRAUD-027:** The system SHALL expose the status of every assertion — Pending, Challenged, Finalized, or Rejected — together with its remaining challenge window, through a public RPC endpoint.

Liveness and censorship resistance

- **REQ-FRAUD-028:** The L1 contract SHALL provide a forced-inclusion inbox through which any user may submit a transaction directly to L1.
- **REQ-FRAUD-029:** A transaction in the forced-inclusion inbox that the sequencer has not included within `FORCED_INCLUSION_TIMEOUT` SHALL become includable by any party, and the sequencer's bond SHALL be subject to slashing for censorship.
- **REQ-FRAUD-030:** If the sequencer fails to produce assertions for longer than `PROPOSER_LIVENESS_TIMEOUT`, any bonded party SHALL be able to propose the next assertion, so that the chain can make progress and users can exit without the sequencer's cooperation.

Determinism

- **REQ-FRAUD-031:** The STF SHALL be bit-for-bit deterministic. It SHALL NOT depend on floating-point arithmetic, wall-clock time, system entropy, uninitialised memory, address-space layout, iteration order of unordered collections, thread scheduling, or any other source of nondeterminism.
- **REQ-FRAUD-032:** The STF SHALL be compiled to a deterministic, fixed instruction-set execution target for which a single-step interpreter can be implemented on Aptos L1.
- **REQ-FRAUD-033:** The build process SHALL be reproducible: a given source revision SHALL produce a byte-identical STF binary, whose hash SHALL be recorded on L1 and referenced by every assertion.
- **REQ-FRAUD-034:** A change to the STF binary hash SHALL follow a governed upgrade path that does not invalidate assertions already pending under the previous hash.

3.1.6 Core L2 Protocol Requirements

- **REQ-CORE-001:** The L2 sequencer SHALL process transactions locally.
- **REQ-CORE-002:** The sequencer SHALL periodically submit a State Root (Merkle Root) representing all accounts, social links, and balances to an Aptos L1 Move contract, in the form of a bonded assertion as defined in REQ-FRAUD-001.
- **REQ-CORE-003:** A State Root committed to Aptos Mainnet SHALL become immutable once the assertion carrying it has finalized in accordance with REQ-FRAUD-004. Prior to finalization, a committed State Root is a *claim*, and may be discarded by a successful challenge.
- **REQ-CORE-004:** Users SHALL be able to use on-chain proof from Aptos L1 to withdraw their funds

directly from the L1 bridge in case of L2 server compromise, against any finalized assertion.

- **REQ-CORE-005:** The system SHALL distinguish, in all APIs and user-facing interfaces, between *soft confirmation* (the transaction has been executed and ordered by the sequencer, achieved in seconds) and *L1 finality* (the covering assertion has finalized, achieved after the challenge window). These SHALL NOT be conflated.
- **REQ-CORE-006:** The protocol SHALL NOT prevent third parties from offering fast-withdrawal liquidity — fronting funds to a user against a pending assertion and assuming the challenge risk in exchange for a fee — so that the challenge window need not be a user-visible delay in the common case.

3.2 Non-Functional Requirements

3.2.1 Performance Requirements

- **NFR-PERF-001:** The L2 network SHALL achieve transaction throughput suitable for consumer applications (e.g., >1,000 transactions per second).
- **NFR-PERF-002:** Transaction finality on the L2 SHALL be achieved within seconds. This refers to soft confirmation as defined in REQ-CORE-005.
- **NFR-PERF-003:** State Root commitments to Aptos L1 SHALL occur every few minutes.
- **NFR-PERF-004:** ZK proof verification SHALL require minimal CPU power to maintain high performance on low-end server infrastructure.
- **NFR-PERF-005:** A verifier node SHALL be able to re-execute batches at a rate exceeding the sequencer's production rate, so that verification does not fall behind the chain.
- **NFR-PERF-006:** A complete dispute, from challenge to on-L1 resolution, SHALL conclude within a bounded time that is strictly less than CHALLENGE_WINDOW.
- **NFR-PERF-007:** One-step verification SHALL complete within a single Aptos L1 transaction, within that chain's gas and computation limits.

3.2.2 Security Requirements

- **NFR-SEC-001:** The system SHALL inherit security from Aptos L1 through cryptographic commitments of the L2 state root.
- **NFR-SEC-002:** User funds SHALL be recoverable directly from Aptos L1 even if the L2 server is compromised.
- **NFR-SEC-003:** All sensitive data, especially personal identifiers, SHALL be stored in a privacy-preserving manner (e.g., cryptographic hashes).
- **NFR-SEC-004:** The Keyless JWT Validation Architecture SHALL prevent exposure of personal information while linking Web2 IDs to blockchain addresses.
- **NFR-SEC-005:** The system SHALL prevent replay attacks for EPK-signed transactions.

- **NFR-SEC-006:** All cryptographic operations SHALL use industry-standard, audited algorithms and libraries.
- **NFR-SEC-007:** The system SHALL be secure under a 1-of-N honest verifier assumption: an invalid state root SHALL NOT finalize so long as at least one honest party runs a verifier node and can reach Aptos L1 within the challenge window.
- **NFR-SEC-008:** No party — including the sequencer operator, the Governance Multi-Signature Council, and the SENA development team — SHALL hold a privileged ability to finalize an assertion, dismiss a challenge, or override the outcome of a dispute.
- **NFR-SEC-009:** SEQUENCER_BOND_MIN SHALL be set such that the cost of forfeiting the bond materially exceeds the value extractable from a successful state transition fraud, and SHALL be reviewable by governance as network value grows.
- **NFR-SEC-010:** The cost to an honest challenger of winning a dispute SHALL be bounded and reimbursed on success, so that an adversary cannot defeat honest verification by attrition.
- **NFR-SEC-011:** The dispute protocol SHALL be resistant to griefing, including spurious challenges intended to delay finalization, and SHALL bound the total delay an adversary can impose per unit of bond committed.
- **NFR-SEC-012:** The fraud proof system, including the L1 dispute contract and the one-step verifier, SHALL undergo independent security audit prior to mainnet launch, and SHALL be treated as in-scope for the network's bug bounty programme at its highest severity tier.

3.2.3 Reliability Requirements

- **NFR-REL-001:** The L2 sequencer SHALL maintain high uptime and availability.
- **NFR-REL-002:** The system SHALL gracefully handle network partitions and temporary L1 unavailability.
- **NFR-REL-003:** Data persistence SHALL be guaranteed through RocksDB, ensuring no loss of state.
- **NFR-REL-004:** The system SHALL retain sufficient execution checkpoints and historical state to construct a One-Step Proof for any pending assertion for the full duration of the challenge window.
- **NFR-REL-005:** A rollback caused by a successful challenge SHALL leave the node in a consistent, recoverable state, with no partial application of discarded batches.

3.2.4 Maintainability Requirements

- **NFR-MAINT-001:** The codebase SHALL be written in Rust, adhering to idiomatic Rust practices and coding standards.
- **NFR-MAINT-002:** The system SHALL be modular, allowing for independent updates and upgrades of its components (e.g., Keyless Auth Engine, Social Connect Index).

- **NFR-MAINT-003:** Comprehensive logging and monitoring capabilities SHALL be integrated to facilitate debugging and operational oversight.
- **NFR-MAINT-004:** The STF executed by the sequencer, by verifier nodes, and by the one-step verifier SHALL derive from a single source of truth, so that the three cannot drift apart.
- **NFR-MAINT-005:** The test suite SHALL include adversarial dispute scenarios — invalid assertions, dishonest challenges, timeout forfeitures, and deep rollbacks — exercised end to end against a local Aptos L1.

3.2.5 Portability Requirements

- **NFR-PORT-001:** The SENA L2 node software SHALL be deployable on standard Linux-based server environments.
- **NFR-PORT-002:** The system SHALL be compatible with various cloud providers and on-premise deployments.
- **NFR-PORT-003:** Bit-for-bit identical execution results SHALL be produced across all supported platforms and CPU architectures, as required by REQ-FRAUD-031.

4. Appendices

4.1 Stakeholder Documentation Excerpt: Network Security Model

SENA is designed to achieve "Aptos-Level Secure" by inheriting the robust security guarantees of the Aptos L1, even when operating on resource-constrained infrastructure. This approach ensures that user funds remain safe from compromise, even if the L2 server itself is physically breached.

A. Inherited Security via Cryptographic Commitment

SENA's L2 server functions as a single sequencer, processing transactions locally. To ensure the integrity and immutability of the L2 state, the sequencer periodically submits a State Root (a Merkle Root representing all accounts, social links, and balances) to a dedicated Aptos L1 Move contract.

The Security Guarantee: Once this State Root is committed to the Aptos Mainnet and has survived its challenge window, it becomes an immutable record. In the event of a compromise or failure of the L2 server, the history of asset ownership is indelibly recorded on the multi-billion-dollar Aptos L1 validator network. Users can leverage this on-chain proof to directly withdraw their funds from the L1 bridge, ensuring no loss of assets.

B. Secure Keyless JWT Validation Architecture

SENA integrates with Aptos Keyless accounts to provide a secure and user-friendly authentication mechanism. This system utilizes Zero-Knowledge (ZK) Proofs to link a user's Web2 identity (e.g., Google JWT) to their blockchain address without exposing sensitive personal information.

Implementation Details:

- **Client-Side Key Pair Generation:** When a user initiates a login via a Web2 provider (e.g., Google, Apple), the client application generates an ephemeral key pair.
- **ZK Proof Request:** The client then requests a ZK proof from a dedicated Prover service. SENA leverages Aptos's public, decentralized prover network for this purpose, offloading the computationally intensive ZK circuit computations from the L2 server.
- **On-Chain Verification:** The SENA L2 node incorporates Aptos's open-source signature verification code, written natively in Rust. Upon receiving a transaction, the sequencer performs a lightweight verification of the pre-computed ZK proof. This verification process requires minimal CPU power compared to generating the proof, allowing SENA to maintain high performance on low-end server infrastructure.

4.2 What Fraud Proofs Change About the Security Model

The commitment scheme described in Appendix 4.1 establishes that the *history* of the L2 is recorded immutably on Aptos L1. On its own, it does not establish that the recorded history is *correct*. A compromised or dishonest sequencer can commit a state root describing a state that no honest execution of the published transactions would ever produce — for example, one in which the attacker's balance has been inflated — and that false root would be recorded on L1 just as immutably as a true one. Users would then withdraw against it, and the L1 bridge would pay out.

The fraud proof system specified in Section 3.1.6 closes this gap through four mechanisms working together:

Mechanism	Requirement	What it prevents
Delayed finalization	REQ-FRAUD-004, REQ-FRAUD-005	Withdrawal against a state root nobody has had the opportunity to check.

Mechanism	Requirement	What it prevents
Mandatory data publication	REQ-FRAUD-007 to REQ-FRAUD-010	A sequencer hiding the transactions, making fraud undetectable in principle.
Permissionless challenge	REQ-FRAUD-011, REQ-FRAUD-024	Verification being the privilege of a closed set that can be captured or coerced.
On-L1 adjudication	REQ-FRAUD-016 to REQ-FRAUD-018	Disputes being settled by a trusted party rather than by Aptos L1 itself.

The resulting guarantee is materially stronger than the one in Appendix 4.1, and it is worth stating precisely, including its cost:

Guarantee. An invalid state root cannot finalize, and therefore cannot be withdrawn against, provided at least one honest party runs a verifier node and can land a transaction on Aptos L1 within the challenge window.

Cost. Funds bridged out to Aptos L1 are subject to the challenge window — seven days by default. This is a real and unavoidable consequence of the optimistic model, not an implementation shortcoming. REQ-CORE-005 and REQ-CORE-006 address how it is presented to users and how third-party liquidity can absorb it in the common case, but the underlying delay is the price of trust minimisation.

Residual assumptions. The guarantee does not hold if every verifier is absent or compromised, if Aptos L1 censors challenge transactions for the full window, if the STF is nondeterministic in a way that makes honest parties disagree (REQ-FRAUD-031), or if a bug in the one-step verifier causes it to adjudicate incorrectly. The first three are addressed by design; the last is why NFR-SEC-012 places the verifier under independent audit.